

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.ensemble import RandomForestClassifier
```

```
df = pd.read_csv('/content/processed-data.csv')
df
```



	Tiredness	Dry-Cough	Difficulty-in-Breathing	Sore-Throat	None_Sympton	Pains	Nasal Congestio
0	1	1	1	1	0	1	
1	1	1	1	1	0	1	
2	1	1	1	1	0	1	
3	1	1	1	1	0	1	
4	1	1	1	1	0	1	
...	...	...	...	...	...	...	.
316795	0	0	0	0	1	0	
316796	0	0	0	0	1	0	
316797	0	0	0	0	1	0	
316798	0	0	0	0	1	0	
316799	0	0	0	0	1	0	

316800 rows x 19 columns

```
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 316800 entries, 0 to 316799
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Tiredness                             316800 non-null  int64
1   Dry-Cough                             316800 non-null  int64
2   Difficulty-in-Breathing               316800 non-null  int64
3   Sore-Throat                           316800 non-null  int64
4   None_Sympton                          316800 non-null  int64
5   Pains                                 316800 non-null  int64
6   Nasal-Congestion                      316800 non-null  int64
7   Runny-Nose                            316800 non-null  int64
8   None_Experiencing                     316800 non-null  int64
9   Age_0-9                               316800 non-null  int64
10  Age_10-19                             316800 non-null  int64
11  Age_20-24                             316800 non-null  int64
12  Age_25-59                             316800 non-null  int64
13  Age_60+                               316800 non-null  int64
14  Gender_Female                         316800 non-null  int64
15  Gender_Male                           316800 non-null  int64
16  Severity_Mild                         316800 non-null  int64
17  Severity_Moderate                     316800 non-null  int64
18  Severity_None                         316800 non-null  int64
dtypes: int64(19)
memory usage: 45.9 MB

```

```
df.describe()
```

```


```

	Tiredness	Dry-Cough	Difficulty-in-Breathing	Sore-Throat	None_Sympton	
count	316800.000000	316800.000000	316800.000000	316800.000000	316800.000000	316800.000000
mean	0.500000	0.562500	0.500000	0.312500	0.062500	
std	0.500001	0.496079	0.500001	0.463513	0.242062	
min	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	0.000000	0.000000	0.000000	0.000000	
50%	0.500000	1.000000	0.500000	0.000000	0.000000	
75%	1.000000	1.000000	1.000000	1.000000	0.000000	
max	1.000000	1.000000	1.000000	1.000000	1.000000	

```
df.head()
```



	Tiredness	Dry-Cough	Difficulty-in-Breathing	Sore-Throat	None_Sympton	Pains	Nasal-Congestion	Ru
0	1	1	1	1	0	1	1	
1	1	1	1	1	0	1	1	
2	1	1	1	1	0	1	1	
3	1	1	1	1	0	1	1	
4	1	1	1	1	0	1	1	

```
df = df.drop(columns=['Severity_Mild', 'Severity_Moderate'])
df.rename(columns={'Severity_None': 'Target'}, inplace=True)
```

```
df.keys()
```



```
Index(['Tiredness', 'Dry-Cough', 'Difficulty-in-Breathing', 'Sore-Throat',
      'None_Sympton', 'Pains', 'Nasal-Congestion', 'Runny-Nose',
      'None_Experiencing', 'Age_0-9', 'Age_10-19', 'Age_20-24', 'Age_25-59',
      'Age_60+', 'Gender_Female', 'Gender_Male', 'Target'],
      dtype='object')
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import pandas as pd
from sklearn.impute import SimpleImputer
import numpy as np
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.linear_model import Lasso
from sklearn.metrics import accuracy_score
```

```
# Decistion tree
from sklearn.tree import DecisionTreeClassifier

dt_classifier = DecisionTreeClassifier(random_state=42)

# Fit the model to the training data
dt_classifier.fit(x_train, y_train)

# Make predictions on the test data
y_pred_dt = dt_classifier.predict(x_test)

# Evaluate the model
mes = mean_squared_error(y_test, y_pred_dt)
print("Mean Squared Error (Decision Tree Classifier):", mes)
accuracy_dt = accuracy_score(y_test, y_pred_dt)
print("Accuracy (Decision Tree Classifier):", accuracy_dt)

print(classification_report(y_test, y_pred_dt))
```

```
⇒ Mean Squared Error (Decision Tree Classifier): 0.05263157894736842
Accuracy (Decision Tree Classifier): 0.9473684210526315
```

	precision	recall	f1-score	support
0	0.91	0.96	0.93	45
1	0.97	0.94	0.96	69
accuracy			0.95	114
macro avg	0.94	0.95	0.95	114
weighted avg	0.95	0.95	0.95	114

```
# logistic model

from sklearn.linear_model import LogisticRegression

logistic_model = LogisticRegression()

# Fit the model to the training data
logistic_model.fit(x_train, y_train)

# Make predictions on the test data
y_pred_logistic = logistic_model.predict(x_test)

# Evaluate the model
accuracy_logistic = accuracy_score(y_test, y_pred_logistic)
print("Accuracy (Logistic Regression):", accuracy_logistic)

print(classification_report(y_test, y_pred_logistic))
```

```
➞ Accuracy (Logistic Regression): 0.956140350877193
              precision    recall  f1-score   support

         0           0.93       0.96       0.95         45
         1           0.97       0.96       0.96         69

 accuracy                   0.96         114
  macro avg              0.95       0.96       0.95         114
 weighted avg            0.96       0.96       0.96         114
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:4
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regr

```
n_iter_i = _check_optimize_result(
```

```
# Random Forest model
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)

# Fit the model to the training data
rf_classifier.fit(x_train, y_train)

# Make predictions on the test data
y_pred_rf = rf_classifier.predict(x_test)

# Evaluate the model
accuracy_rf = accuracy_score(y_test, y_pred_rf)
print("Accuracy (Random Forest Classifier):", accuracy_rf)

print(classification_report(y_test, y_pred_rf))
```

```
⇒ Accuracy (Random Forest Classifier): 0.9824561403508771
```

	precision	recall	f1-score	support
0	0.96	1.00	0.98	45
1	1.00	0.97	0.99	69
accuracy			0.98	114
macro avg	0.98	0.99	0.98	114
weighted avg	0.98	0.98	0.98	114

```

from sklearn.naive_bayes import GaussianNB
nb_classifier = GaussianNB()

# Fit the model to the training data
nb_classifier.fit(x_train, y_train)

# Make predictions on the test data
y_pred_nb = nb_classifier.predict(x_test)

# Evaluate the model
accuracy_nb = accuracy_score(y_test, y_pred_nb)
print("Accuracy (Naive Bayes):", accuracy_nb)

print(classification_report(y_test, y_pred_nb))

```

```

➡ Accuracy (Naive Bayes): 0.9649122807017544

```

	precision	recall	f1-score	support
0	0.96	0.96	0.96	45
1	0.97	0.97	0.97	69
accuracy			0.96	114
macro avg	0.96	0.96	0.96	114
weighted avg	0.96	0.96	0.96	114

```

import matplotlib.pyplot as plt
import seaborn as sns

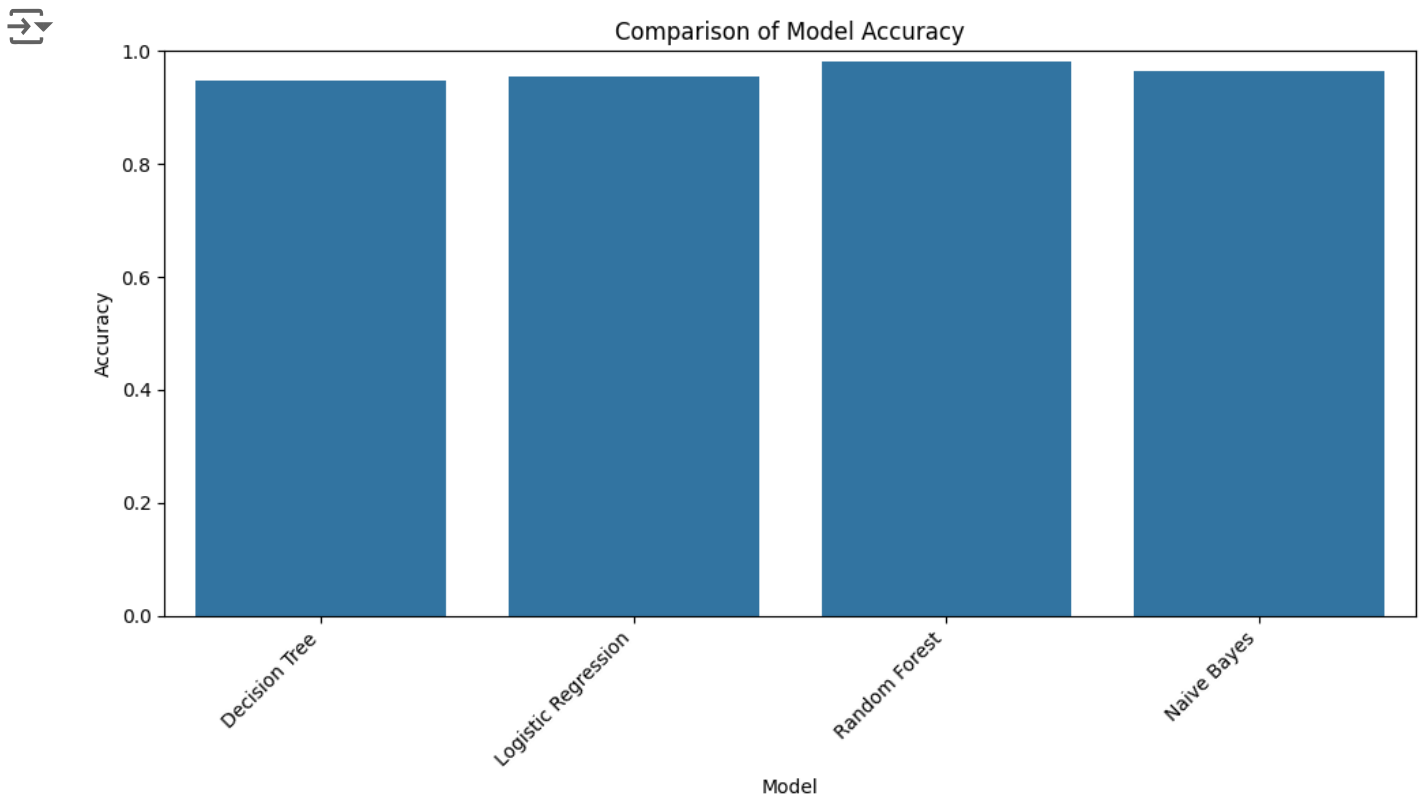
# Store accuracy scores in a dictionary
# Make sure all values are single floats, not lists
model_accuracy = {

    'Decision Tree': accuracy_dt,
    'Logistic Regression': accuracy_logistic,
    'Random Forest': accuracy_rf,
    'Naive Bayes': accuracy_nb
}

# Create a bar plot to visualize accuracy scores
plt.figure(figsize=(10, 6))
sns.barplot(x=list(model_accuracy.keys()), y=list(model_accuracy.values()))
plt.xlabel('Model')
plt.ylabel('Accuracy')

```

```
plt.title('Comparison of Model Accuracy')
plt.ylim(0, 1) # Set y-axis limits for better visualization
plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for readability
plt.tight_layout()
plt.show()
```



```
from sklearn.model_selection import cross_val_score

# Decision Tree Classifier
dt_scores = cross_val_score(dt_classifier, x, y, cv=5)
print("Decision Tree Classifier Cross-Validation Accuracy:", dt_scores.mean())

# Logistic model
logistic_scores = cross_val_score(logistic_model, x, y, cv=5)
print("Logistic Regression Cross-Validation Accuracy:", logistic_scores.mean())

# Random Forest Classifier
rf_scores = cross_val_score(rf_classifier, x, y, cv=5)
print("Random Forest Classifier Cross-Validation Accuracy:", rf_scores.mean())

# Naive Bayes
nb_scores = cross_val_score(nb_classifier, x, y, cv=5)
```



```
print("Naive Bayes Cross-Validation Accuracy:", nb_scores.mean())
```

➡ Decision Tree Classifier Cross-Validation Accuracy: 0.9173420276354604
 /usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:4
 STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regr

```
n_iter_i = _check_optimize_result(
```

/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:4

STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regr

```
n_iter_i = _check_optimize_result(
```

/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:4

STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regr

```
n_iter_i = _check_optimize_result(
```

/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:4

STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regr

```
n_iter_i = _check_optimize_result(
```

/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:4

STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regr

```
n_iter_i = _check_optimize_result(
```

Logistic Regression Cross-Validation Accuracy: 0.943766495885732

Random Forest Classifier Cross-Validation Accuracy: 0.9560937742586555

Naive Bayes Cross-Validation Accuracy: 0.9385188635305075

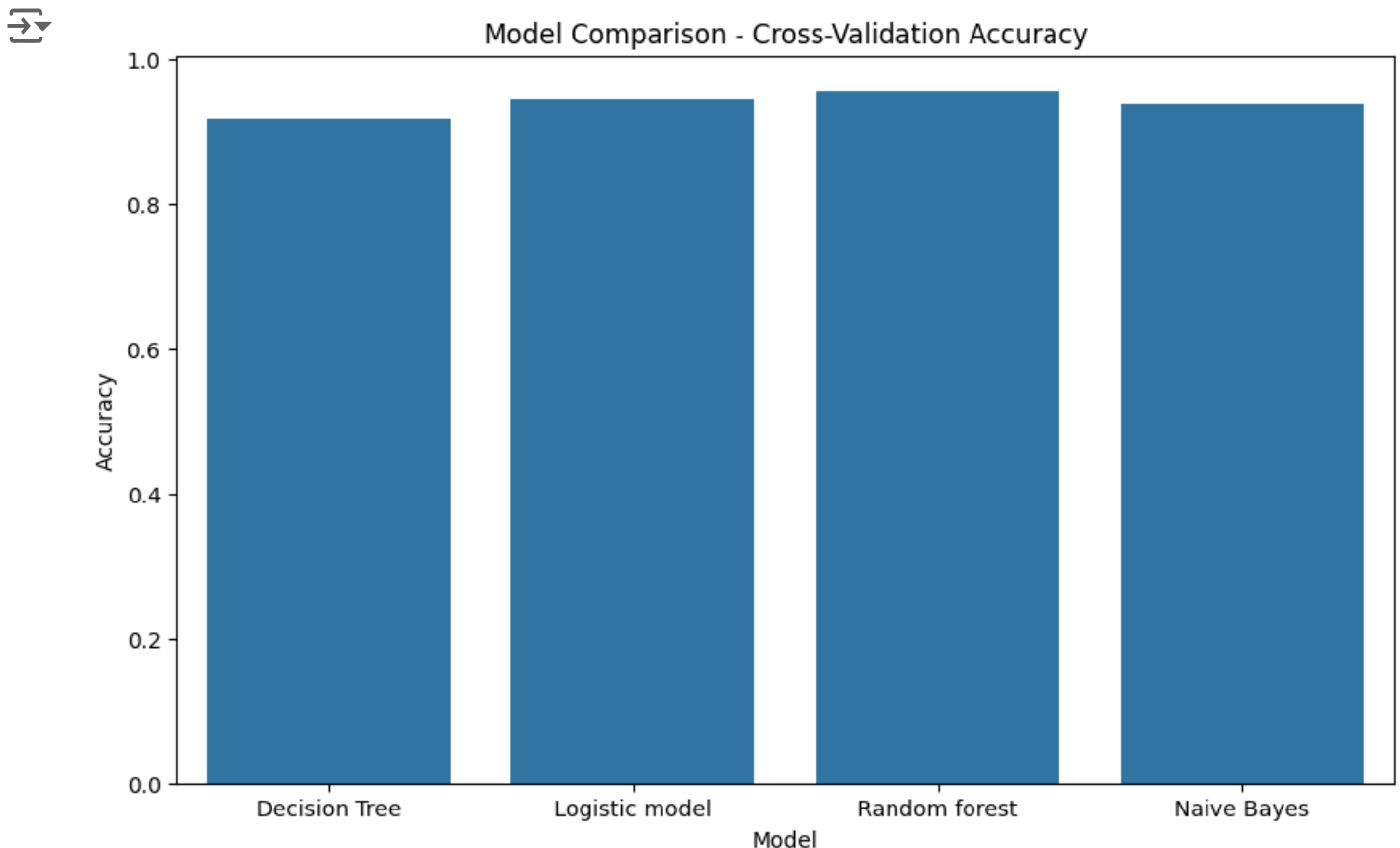
```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.tree import plot_tree
from sklearn import tree

# Comparison DataFrame
models = [ 'Decision Tree' , ' Logistic model ' , ' Random forest ' , 'Naive Bay
accuracies = [ dt_scores.mean(), logistic_scores.mean(), rf_scores.mean(), nb_s

comparison_df = pd.DataFrame({'Model': models, 'Accuracy': accuracies})

# Bar Plot for Comparison
plt.figure(figsize=(10, 6))
sns.barplot(x='Model', y='Accuracy', data=comparison_df)
plt.title('Model Comparison - Cross-Validation Accuracy')
plt.xlabel('Model')
plt.ylabel('Accuracy')
plt.show()
```



```
import time

# Initialize dictionaries to store training and evaluation times
training_times = {}
evaluation_times = {}

# Decision Tree Classifier
start_time = time.time()
dt_classifier.fit(x_train, y_train)
training_times['Decision Tree'] = time.time() - start_time

start_time = time.time()
y_pred_dt = dt_classifier.predict(x_test)
evaluation_times['Decision Tree'] = time.time() - start_time

# Logistic Regression
start_time = time.time()
logistic_model.fit(x_train, y_train)
training_times['Logistic Regression'] = time.time() - start_time

start_time = time.time()
y_pred_logistic = logistic_model.predict(x_test)
evaluation_times['Logistic Regression'] = time.time() - start_time

# Random Forest Classifier
start_time = time.time()
rf_classifier.fit(x_train, y_train)
training_times['Random Forest'] = time.time() - start_time

start_time = time.time()
y_pred_rf = rf_classifier.predict(x_test)
evaluation_times['Random Forest'] = time.time() - start_time

# Naive Bayes
start_time = time.time()
nb_classifier.fit(x_train, y_train)
training_times['Naive Bayes'] = time.time() - start_time

start_time = time.time()
y_pred_nb = nb_classifier.predict(x_test)
evaluation_times['Naive Bayes'] = time.time() - start_time

# Create a DataFrame for visualization
time_df = pd.DataFrame({
    'Model': list(training_times.keys()),
    'Training Time': list(training_times.values()),
```

```
'Evaluation Time': list(evaluation_times.values())
})

# Melt the DataFrame for easier plotting
time_df_melted = pd.melt(time_df, id_vars=['Model'], var_name='Time Type', valu

# Plot the training and evaluation times
plt.figure(figsize=(10, 6))
sns.barplot(x='Model', y='Time (seconds)', hue='Time Type', data=time_df_meltec
plt.title('Training and Evaluation Time Comparison')
plt.xlabel('Model')
plt.ylabel('Time (seconds)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
→ /usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:4  
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

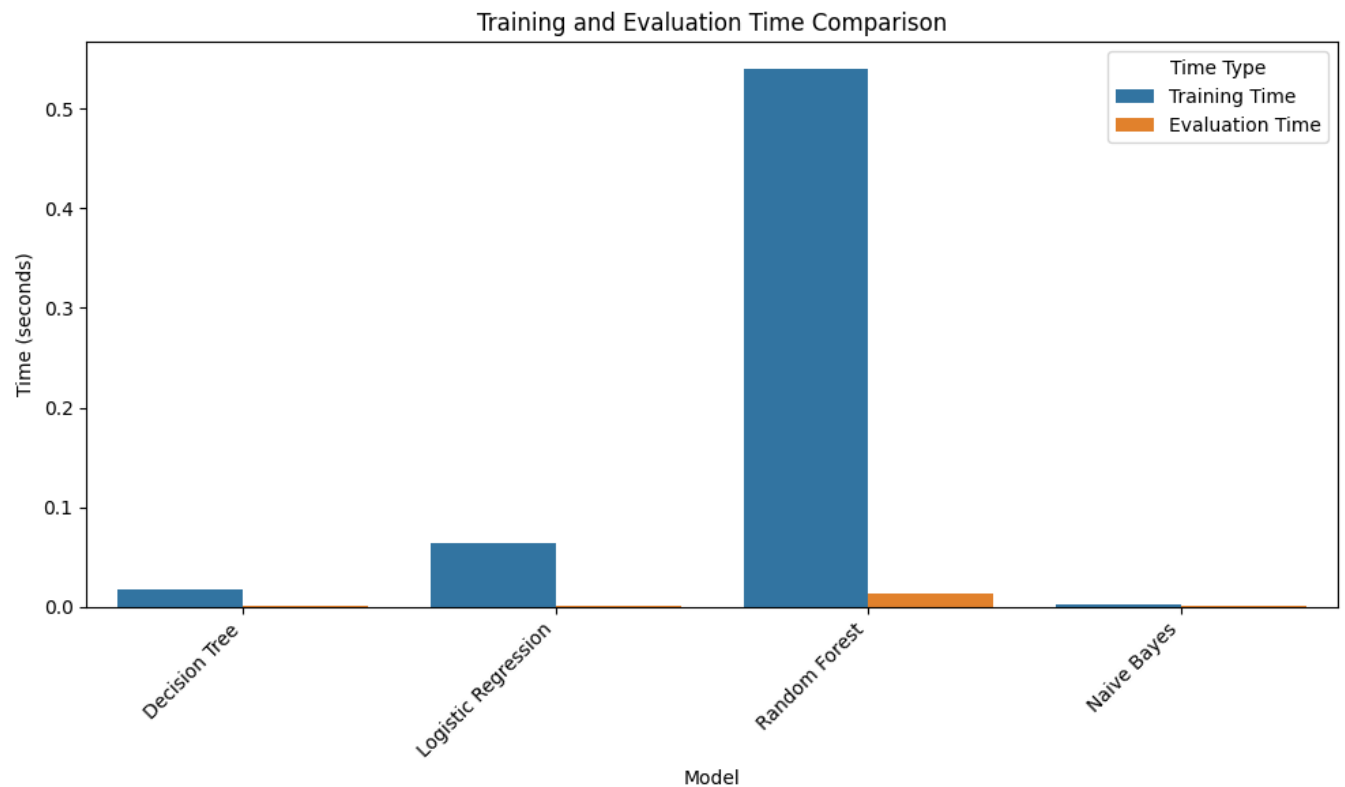
Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regr

```
n_iter_i = _check_optimize_result(
```



```
from sklearn.preprocessing import StandardScaler

# Initialize a StandardScaler object
scaler = StandardScaler()

# Fit the scaler to the training data and transform both training and testing c
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)

# Decision Tree Classifier (with scaling – usually not necessary for tree-based)
dt_classifier_scaled = DecisionTreeClassifier(random_state=42)
dt_classifier_scaled.fit(x_train_scaled, y_train)
y_pred_dt_scaled = dt_classifier_scaled.predict(x_test_scaled)
accuracy_dt_scaled = accuracy_score(y_test, y_pred_dt_scaled)
print("Accuracy (Scaled Decision Tree Classifier):", accuracy_dt_scaled)

# Logistic Regression (with scaling)
logistic_model_scaled = LogisticRegression()
logistic_model_scaled.fit(x_train_scaled, y_train)
y_pred_logistic_scaled = logistic_model_scaled.predict(x_test_scaled)
accuracy_logistic_scaled = accuracy_score(y_test, y_pred_logistic_scaled)
print("Accuracy (Scaled Logistic Regression):", accuracy_logistic_scaled)

# Random Forest Classifier (with scaling – usually not necessary for tree-based)
rf_classifier_scaled = RandomForestClassifier(n_estimators=100, random_state=42)
rf_classifier_scaled.fit(x_train_scaled, y_train)
y_pred_rf_scaled = rf_classifier_scaled.predict(x_test_scaled)
accuracy_rf_scaled = accuracy_score(y_test, y_pred_rf_scaled)
print("Accuracy (Scaled Random Forest Classifier):", accuracy_rf_scaled)

# Naive Bayes ( with scaling )
nb_classifier_scaled = GaussianNB()
nb_classifier_scaled.fit(x_train_scaled, y_train)
y_pred_nb_scaled = nb_classifier_scaled.predict(x_test_scaled) # Use the fitted
accuracy_nb_scaled = accuracy_score(y_test, y_pred_nb_scaled)
print("Accuracy (Scaled Naive Bayes):", accuracy_nb_scaled)
```

```
➞ Accuracy (Scaled Decision Tree Classifier): 0.9473684210526315
Accuracy (Scaled Logistic Regression): 0.9824561403508771
Accuracy (Scaled Random Forest Classifier): 0.6140350877192983
Accuracy (Scaled Naive Bayes): 0.9649122807017544
```

```
# Store accuracy scores in a dictionary for scaled models
model_accuracy_scaled = {

    'Decision Tree (Scaled)': accuracy_dt_scaled,
    'Logistic Regression (Scaled)': accuracy_logistic_scaled,
    'Random Forest (Scaled)': accuracy_rf_scaled,
    'Naive Bayes (Scaled)': accuracy_nb_scaled
}

# Create a bar plot to visualize accuracy scores for scaled models
plt.figure(figsize=(12, 6))
sns.barplot(x=list(model_accuracy_scaled.keys()), y=list(model_accuracy_scaled.values()))
plt.xlabel('Model')
plt.ylabel('Accuracy')
plt.title('Comparison of Model Accuracy (After Scaling)')
plt.ylim(0, 1) # Set y-axis limits for better visualization
plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for readability
plt.tight_layout()
plt.show()
```

